Agenda

- Javascript Introduction
- Language Fundamentals
- Functions and their types
- Object

Variable Scope

1. global
    - a variable declared outside any function
    - can be declared with or without var keyword
    - can be accessed outside or inside any function
    - E.g.

```
num = 100;
var salary = 4.5;
```

    - can be declared inside a function without using a var keyword
    - E.g.

```
function function1() {
// global
firstName = “test”;
}
```

2. Local
    - Must be declared inside a function with keyword var
    - Can NOT be accessed outside the function in which it is declared
    - E.g.

```
function function1() {
// local
var firstName = “test”;
}
```

3. Block
    - Declared with "const" or "let" keyword.
    - Can NOT be accessed outside the block in which it is declared
    - E.g

```
function function1() {
   const firstName = "test";
```

SUNBEAM INFOTECH

```
}
    console.log(firstName); // error - not accessible outside the block
```

2. Complex/Reference Data Types (object)

- Unlike primitives, objects are stored by reference.
- Anything that is not a primitive type in JavaScript falls under the object umbrella.

1. Standard Objects
    - Written in curly braces {} to store keyed collections.
2. Arrays
    - Written in square brackets [], arrays are just a special type of object used to store ordered lists.
3. Functions
    - Functions are also technically objects in JavaScript, meaning they can have properties and can be passed around like values.

null vs undefined

null

- null represents an intentional absence of any object value, while undefined represents a variable that has been declared but not yet given a value.
- null is used when you want to show that a variable is currently empty on purpose.
- used to reset an object reference or clear out old data.

undefined

- A variable automatically becomes undefined when you declare it with let or var without giving it a value.
- JS returns undefined when you try to access an object property or a function argument that does not exist.

Strict Mode

- "use strict"; directive defines that JavaScript code should be executed in "strict mode".
- The "use strict" directive was new in ECMAScript version 5.
- With strict mode good practices are enforced e.g. use of undeclared/implicit variables raise error.
- You can use strict mode in all your programs.
- It helps you to write cleaner code, like preventing you from using undeclared variables. Supported in all modern browsers.
- Not supported in IE-9 and earlier.

Template Strings

- strings can be enclosed in '..', "..", or `` .
- templated strings to print var values: var=${varname}

SUNBEAM INFOTECH

```
 console.log(`addition: ${n1} + ${n2} = ${res}`);
```

JS Operators

- Basic operators are same as C/Java.
- == operator compares the contents (but not the data type).
- === operator compares the contents as well as data types.
- Example

```
console.log(`123 == "123" : `, 123 == "123"); // true
console.log(`123 === "123" : `, 123 === "123"); // false
```

- similarly != compare contents, while !== compraes contents + data types.
- Logical && returns first falsy value (this may be false, 0, empty string, or null)
- Logical || returns first truthy value (this may be true, non-zero number, non-empty string, or non-null object)
- falsy values: 0 (number), false (boolean), null (object), '' (string), undefined
- Example:

```
console.log(`0 && 12 : `, 0 && 12); // first false value
console.log(`0 || 12 : `, 0 || 12); // first true value
```

Function

- function is a reusable block of statement
- defined once & can be called multiple times.
- Function must be declared along with its definition
- To declare a function, function keyword is used
- Syntax: function ( ) { // function body }

Types of function

1. function may or may not have arguments (input to fn).

```
 // function with no arguments
function printline() {
console.log("----------------------------------------");
}

// function with arguments
function multiply(a, b) {
const c = a * b;
```

SUNBEAM INFOTECH

```
console.log("multiply result = " + c);
}
```

2. function may or may not return result (output from fn).

```
// function returning result
function subtract(a, b) {
const c = a - b;
return c;
}
// function call
let res = subtract(22, 7);
```

Function Argument types

1. function argument types are inferred. fn can be called multiple times with different types of args.

```
// function arguments are not type-safe
function add(x, y) {
    const z = x + y;
    console.log(`${x} + ${y} = ${z} of type ${typeof z}`);
}

// function can be called with different arg types
add(22, 7);
add("Hello", "World");
add(10, true);
add("Bond", 7);
add("Flag", false);

// passing more arguments will ignore remaining args.
add(10, 20, 30);

// passing less arguments will consider remaining args undefined
add(10);
```

SUNBEAM INFOTECH

2. if multiple fns defined with same name, last fn definition override/replace the earlier ones.

```
function f1() {
console.log("f1() called.");
}
function f1(x, y) {
console.log("f1(x, y) called: " + x + ", " + y);
}
function f1(x) {
```

```
console.log("f1(x) called: " + x);
}
f1(10, 20); // calls last f1(x).
```

3. By default, if any arg is not passed, it is undefined. Instead default value can be provided a.k.a. "default parameters".

```
// Arguments are assigned to params from left to right (irrespective of default values given to params).

// function definition
function add(x = 0, y, z = 0) {
let r = x + y + z;
console.log(`add(): ${x} + ${y} + ${z} = ${r}`);
}
// function calls
add();
add(10);
add(22, 7);
add(11, 22, 33);
```

Built-In Functions

- JS have built-in functions as well.
- e.g. parseInt(), parseFloat(), eval()

```
// string to number (int)
let n1 = parseInt("123");
console.log("n1 = " + n1); // 123

let n2 = parseInt("123.45");
console.log("n2 = " + n2); // 123

let n3 = parseInt("Bond007");
console.log("n3 = " + n3); // NaN

let n4 = parseInt("007Bond");
console.log("n4 = " + n4); // 7

 // string to number (float)
let v1 = parseFloat("3.14");
console.log("v1 = " + v1); // 3.14

let v2 = parseFloat("123");
console.log("v2 = " + v2); // 123

// eval() to evaluate string expr
```

SUNBEAM INFOTECH

```
let v3 = eval("2 + 3 * 4");
console.log("v3 = " + v3); // 14
```

Function Alias

- In JS, function can be treated as object.
- It can be assigned to another reference a.k.a. function alias.
- Using alias the actual function can be called.
- Function alias concept can be used to pass fn as argument to fn, Return fn from another fn, Create anonymous fns and Lambda expressions.
- E.g.

```
// function definition
function add(x, y) {
let z = x + y;
console.log(`add(): ${x} + ${y} = ${z}`);
}
console.log("add() -- type = " + typeof add + ", name = " + add.name);
add(22, 7);

// function alias
const calc = add;
console.log("calc -- type = " + typeof calc + ", name = " + calc.name);
calc(23, 8);
```

- Anonymous functions are created using function keyword, but not given any name.
- Anonymous fns assigned to alias and called via aliases only.
- Function without a name is called as anonymous function
- E.g.

```
// anonymous function
const subtract = function (x, y) {
let z = x - y;

console.log(`subtract(): ${x} - ${y} = ${z}`);
};
subtract(22, 7);
```

SUNBEAM INFOTECH

Arrow Functions

- Arrow functions are a modern, concise way to write function expressions in JavaScript, introduced in ECMAScript 2015 (ES6).
- They provide a shorter syntax than traditional function expressions, making the code more readable, especially for simple, one-line functions.
- An arrow function has three main parts:
    1. Parameters: Enclosed in parentheses ().

2. The "fat arrow": The => syntax.
3. Function body: The code to be executed.

- They can be one-liner or multiline. However, one-liners are preferred.

```
const add = (a, b) => {
    const res = a + b
    console.log(res)
}

// Omitting parentheses for a single parameter:
const square = n => {
    const res = n * n
    console.log(res)
}

//Omitting curly braces and return for a single expression: For one-line arrow functions, the return is implicit.
const square = x => x * x;

//Returning an object literal: To implicitly return an object, you must wrap the curly braces in parentheses to avoid a syntax error.
const createObject = () => ({ message: "Hello" });
```

- If the function body has multiple lines, you must use curly braces and an explicit return statement.

Closure

- A closure in JS is the combination of a function and the lexical environment within which that function was declared.
- This environment consists of any variables that were in scope at the time the function was created.
- Closures allow a function to access variables from its outer scope even after that scope has finished executing

```
 // function closure
 function outer() {
let z = 1;
// inner/nested fn -- accessing local var of outer fn
function inner(x, y) {
return x + y + z;
  }
return inner;
}

// call outer fn & collect its return value
 const infn = outer();
// call inner fn
 const result = infn(22, 7);
console.log("final closure result = " + result);
```

SUNBEAM INFOTECH

```
function counter() {
    let count = 0
    return () => {
        count++;
        return count;
    }
}
const incrementCounter = counter()

console.log(incrementCounter())
console.log(incrementCounter())
console.log(incrementCounter())
```

Currying

- Currying is an advanced functional programming technique that transforms a function with multiple arguments into a sequence of nested functions, each taking a single argument.
- Instead of calling a function like f(a, b, c), a curried function is called as f(a)(b)(c).
- This process is made possible in JavaScript through the use of closures, which allow the inner functions to retain access to the arguments of their outer functions' scopes.
- Benefits of currying in JavaScript

1. Partial application: Currying allows for partial application of functions, enabling developers to create specialized versions of functions by fixing some arguments.
2. Composability: Curried functions are highly composable, making it easier to build complex operations by combining simpler functions.
3. Code reuse: Currying promotes code reuse by breaking down functions into smaller units that can be reused in various contexts.

- Without Currying using multiple functions

SUNBEAM INFOTECH

```
function HrsToMin(n) {
    return n * 60
}

function MinToSec(n) {
    return n * 60
}

function KmToM(n) {
    return n * 1000
}

// Functions needs to be defined seperately first to be called
console.log(HrsToMin(2))
console.log(MinToSec(20))
console.log(KmToM(5))
```

- Without Currying using single functions

```
// Function becomes a generic function
function multiply(factor, value) {
    return factor * value
}

// every time function needs to be called passing all the arguments
console.log(multiply(60,2))
console.log(multiply(60,3))

console.log(multiply(60,20))
console.log(multiply(60,30))

console.log(multiply(1000,5))
```

- Using currying

```
// Allows to  create specialized function as per the requirement
function multiply(factor) {
    return (value) => {
        return value * factor
    }
}

// Currying allows to create multiple different functions if required
const HrsToMin = multiply(60)
const MinToSec = multiply(60)

console.log(HrsToMin(2))
console.log(HrsToMin(3))
```

SUNBEAM INFOTECH

```
console.log(MinToSec(20))
console.log(MinToSec(30))

// If function is required only once
console.log(multiply(1000)(5))
```

Higher Order Functions

- A higher-order function (HOF) is a function that either takes one or more functions as arguments, returns a function, or both.
- This is possible in JavaScript because functions are "first-class citizens," meaning they can be treated like any other value (such as a string or number).
- A function that is passed as an argument to a higher-order function is known as a callback function.
- This is a common pattern used to handle asynchronous operations and create reusable code.

```
//executer function
// Higher Order functions
function executer(n1, n2, fn) {
    const res = fn(n1, n2)
    console.log('res - '+res)
}

const add = (n1,n2)=>{
    const res = n1+n2;
    return res;
}
// Here add is called as callback function
executer(10,20,add)

// we can pass the callback function by directly declaring it in the argument
executer(10,20,(n1,n2)=>n1-n2)
executer(10,20,(n1,n2)=>n1*n2)
```

Function Hidden Parameters

- Every function in JS accepts 2 hidden parameters

1. this:
    - refers the current object on which the function is called
    - if function is called on an instance, the instance becomes this
    - E.g.

```
var p = new Object
p.myCanVote = canVote;
```

SUNBEAM INFOTECH

```
// the variable p becomes this inside canVote()
p.myCanVote()
```

- if function is called without an instance then Window becomes this

```
function canVote() { /* body */}

// inside canVote Window becomes this
canVote();
```

2. arguments:

- is of type array
- array of parameter values passed to a function
- E.g.

```
function add() {
var answer = 0;
for (var index = 0; index < arguments.length; index++) {
answer += arguments[index];
}
console.log(answer)
}
```

Difference between arrow functions and normal function

- Arrow functions are a modern, more concise alternative introduced in ES6, while normal functions have been part of JavaScript since its inception.
- this keyword behavior : This is the most significant difference between the two function types.
- Normal functions: The value of this is dynamic and depends on how the function is called.
  - As an object method: this refers to the object that owns the method.
  - As a standalone function: this defaults to the global object (window in browsers) or is undefined in strict mode.
- Arrow functions: They do not have their own this binding. Instead, they inherit the this value from the parent or enclosing lexical scope where they were defined. This behavior is predictable and avoids common issues with this losing its context in callbacks.

```
const user = {
  name: 'Alex',
  // Normal function: 'this' refers to the 'user' object
  regularMethod: function() {
    console.log(this.name); // Logs: Alex
  },
  // Arrow function: 'this' is inherited from the global scope
  arrowMethod: () => {
```

SUNBEAM INFOTECH

```
    console.log(this.name); // Logs: undefined
  }
};

user.regularMethod();
user.arrowMethod();
```

Hoisting

- In C/C++, variables or functions or classes must be declared before their use.
- Hoisting in JavaScript is a behavior where variable and function declarations are moved to the top of their scope before code execution.
- It allows using variables and functions before they are declared in the code. However, only declarations are hoisted, not initializations or assignments.

Variable Hoisting

- Variables declared with var are hoisted to the top of their scope and initialized with undefined.

```
    console.log(x); // Output: undefined
    var x = 5;
    console.log(y); // Output: ReferenceError: Cannot access 'y' before
initialization
    let y = 10;
```

Function Hoisting

- Function declarations are fully hoisted, allowing them to be called before they are defined in the code.

```
    greet("Sunbeam"); // Output: Hello, Sunbeam!
    function greet(name) {
        console.log("Hello, " + name + "!");
    }
```

Object

- Everything in JS is an Object, even functions are also objects
- Object is a collection of properties (data members or fields) and methods
- Objects are dynamic, meaning you can add, modify, or delete properties after they are created.
- Properties can be accessed/added using dot operator or [] subscript.
- To create an object (instance) in JS, we can use
    1. { } (Object Literal)
    2. new Object()
    3. constructor function

1. { } Using Object Literal

SUNBEAM INFOTECH

- Create object using JSON syntax.

```
 // create object & props in JS -- (using {})
let i = {
name: "Pencil",
company: "Natraj",
price: 5.0,
};

console.log("type of i = " + typeof(i));
console.log("i name = ", i.name);
console.log("i company = ", i.company);
console.log("i price = ", i["price"]);
```

2. Using new Object().

- Create object using new Object()
- Properties can be added dynamically or accessed using dot/[] operator.

```
// create object & props in JS -- (using .)
let p = new Object();
p.name = "James Bond";
p.age = 65;
p.addr = "London";
console.log("type of p = " + typeof(p));
console.log("p name = ", p.name);
console.log("p age = ", p.age);
console.log("p addr = ", p.addr);

// create object & props in JS -- (using [])
let m = new Object();
m["model"] = "iPhone 14 Pro";
m["company"] = "Apple";
m.price = 89000.0;
console.log("type of m = " + typeof (m));
console.log("m model = ", m["model"]);
console.log("m company = ", m.company);
console.log("m price = ", m["price"]);
```

SUNBEAM INFOTECH

3. Using contructor function

- Create object using constructor function i.e. new FnName(args).
- Properties are added into fn using "this" pointer (which refers to newly created object).
- To add common properties & methods for all objects use prototype

```
// using construction function
// Starting a constructor function name with a capital letter is a naming
```

```
convention, not a technical requirement enforced by the JavaScript engine.
// Helps to Distinguish from regular functions
function Car(model, company, price) {
this.model = model;
this.company = company;
this.price = price;
}

// add properties and methods for all objects using "prototype".
Car.prototype.color = "red";
Car.prototype.printInfo = function () {
console.log("car model = ", this.model);
console.log("car company = ", this.company);
console.log("car price = ", this.price);
console.log("car color = ", this.color);
};

// Create Car object
let c1 = new Car("i10", "Hundai", 800000.0);
c1.printInfo();
// Create another Car object
let c2 = new Car("Punch", "Tata", 900000.0);
c2.printInfo();
```

SUNBEAM INFOTECH